Nonparametric Methods

EECS 492

Introduction to Artificial Intelligence

Today's Class

- Non-parametric methods
  - kNN
  - SVM

Parametric vs. Nonparametric ML

- Parametric models
  - Goal: estimate a fixed set of parameters $\boldsymbol{w}$
  - Once parameters are estimated – toss training data
  - Important idea: the number of parameters is fixed
  - Examples: linear regression, neural networks

- Nonparametric models
  - Important idea: the number of parameters varies
  - Example: instance-based learning, table lookup

Table Lookup

- Very very simple method of classification
  - Given: table of x values (features) → training data
  - Given: query feature, x_i → test data point
  - Assign a y_i from table corresponding to a specific feature value, x_i

- What is our potential problem here?
  - Does this method scale well to large feature spaces?
  - What about large data sets?
  - What could we do to improve the situation?

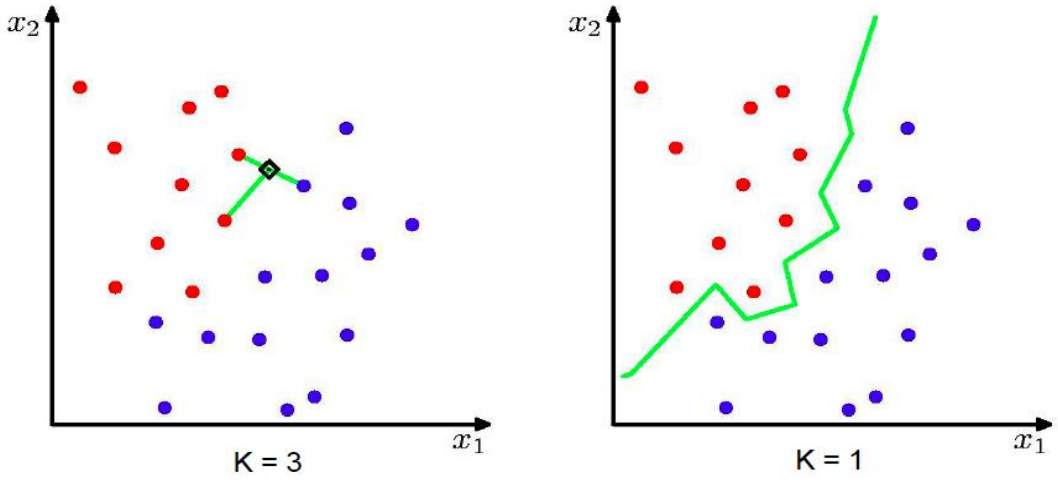
K-Nearest Neighbors (k-NN)

Nearest Neighbor Method

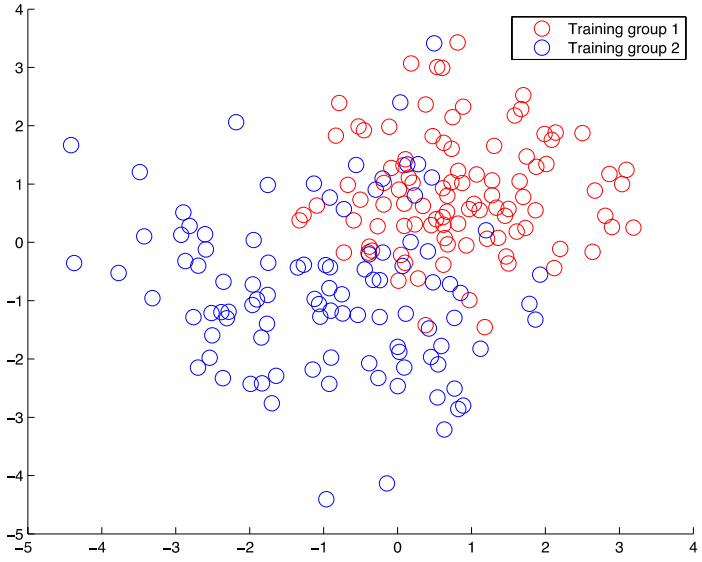
- What if we weren't concerned with an exact match?

- k-Nearest Neighbors
  - Given: query $\mathbf{x}_q$
  - Find: k examples nearest to $\mathbf{x}_q$

- Approach
  - Find the $NN(k,x_q)$
  - Take a plurality vote of neighbors
  - Assign label to class with majority vote

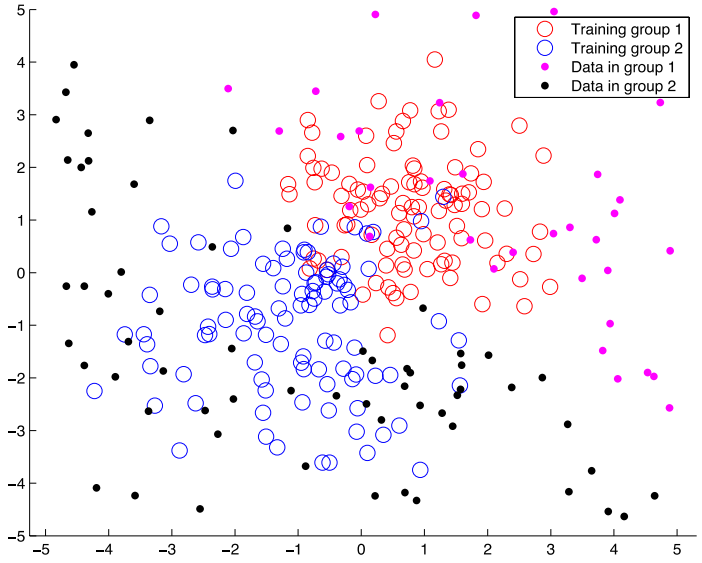


k-Nearest Neighbors – Example

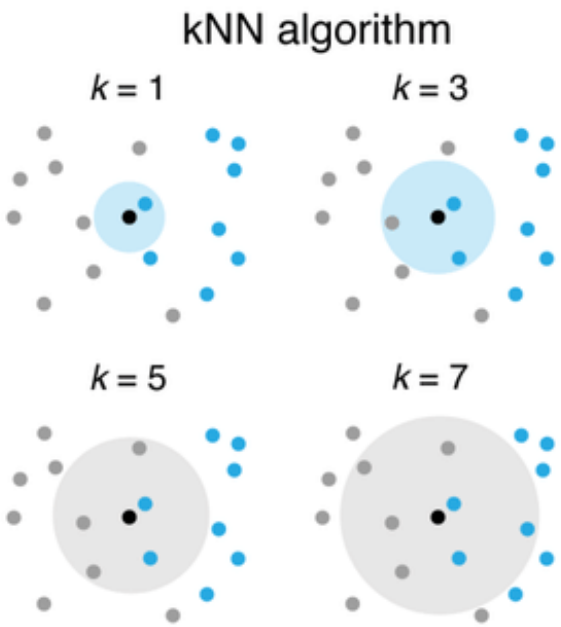


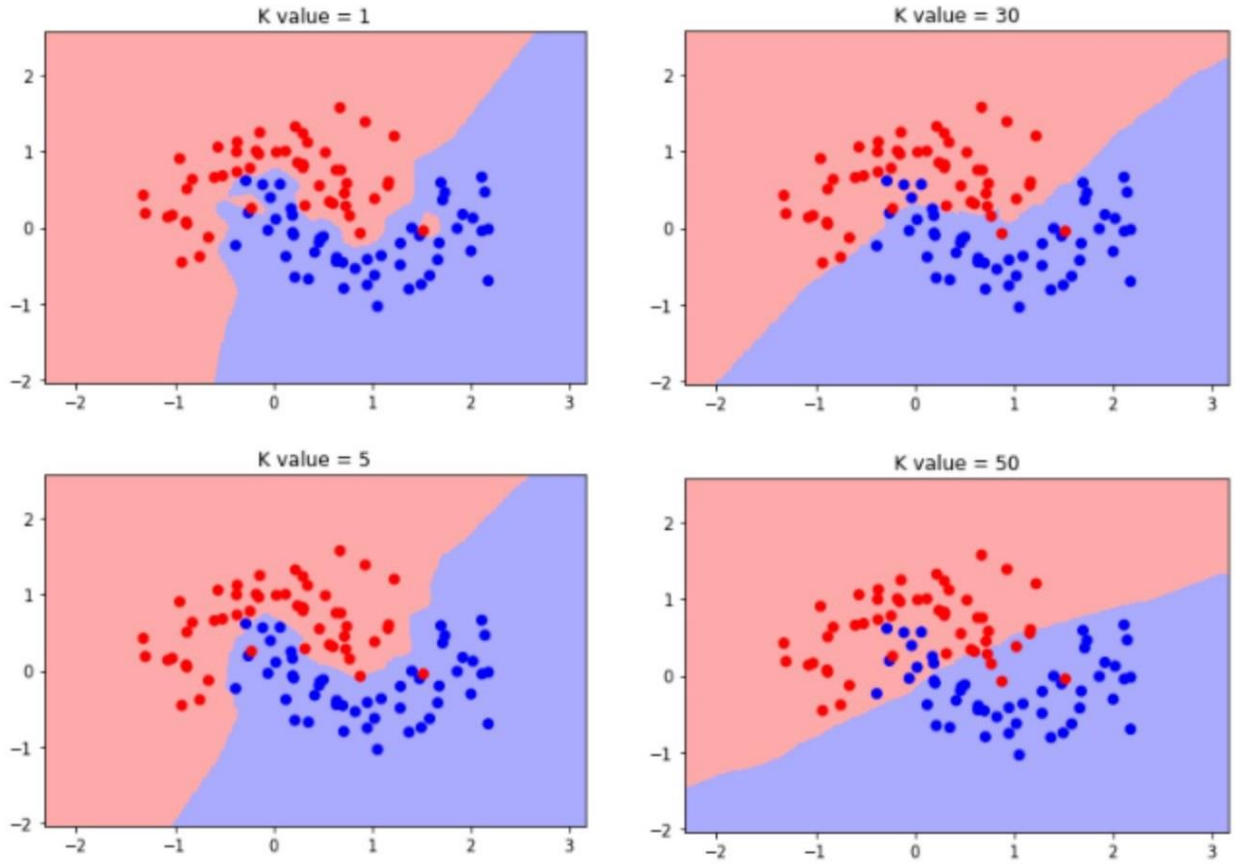
The original training data

Unlabeled testing data assigned based on k-Nearest Neighbors



k-Nearest Neighbors – Effect of k

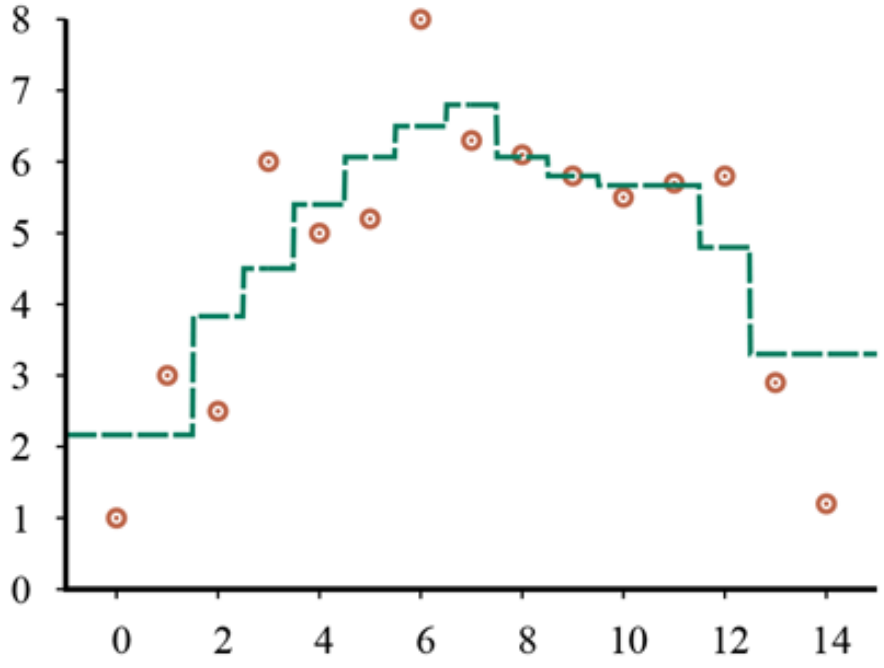




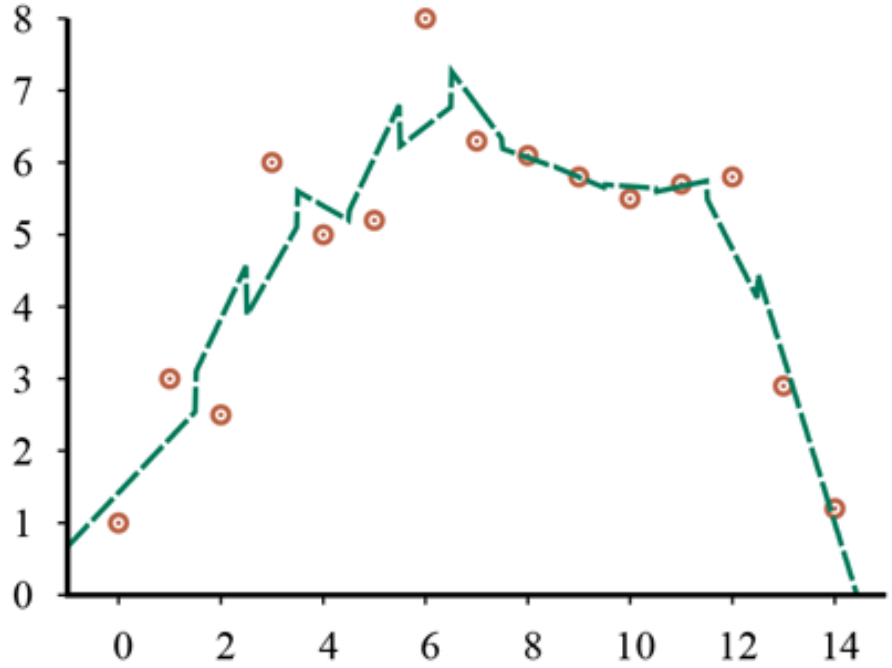
How should we calculate distance?

What about Regression?

3-nearest neighbors average



3-nearest-neighbors linear regression

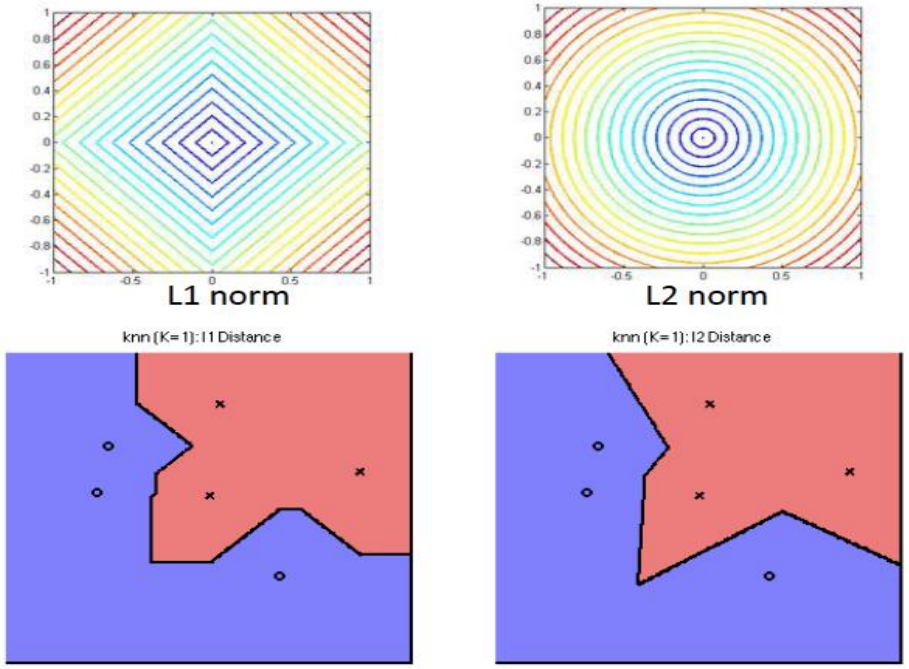


Distance Calculation in k-NN

- Most common distance metric: Minkowski distance (L^P norm)

$$L^P(x_j, x_q) = \left(\sum_i |x_{j,i} - x_{q,i}|^p\right)^{1/p}$$

  - When $p = 1$ – Manhattan distance
  - When $p = 2$ – Euclidean distance



Normalization in k-NN

- Different features (dimensions) could have totally different units!
  - E.g,. one feature is measured in miles, one measured in meters
  - Changing unit of a feature changes the distance calculation

- Common to apply Z-normalization in each dimension
  - Compute the mean μ_i and standard deviation σ_i of the values in each dimension I
  - Rescale each value: $x_{j,i}$ becomes $(x_{j,i} - \mu_i)/\ \sigma_i$

Curse of Dimensionality

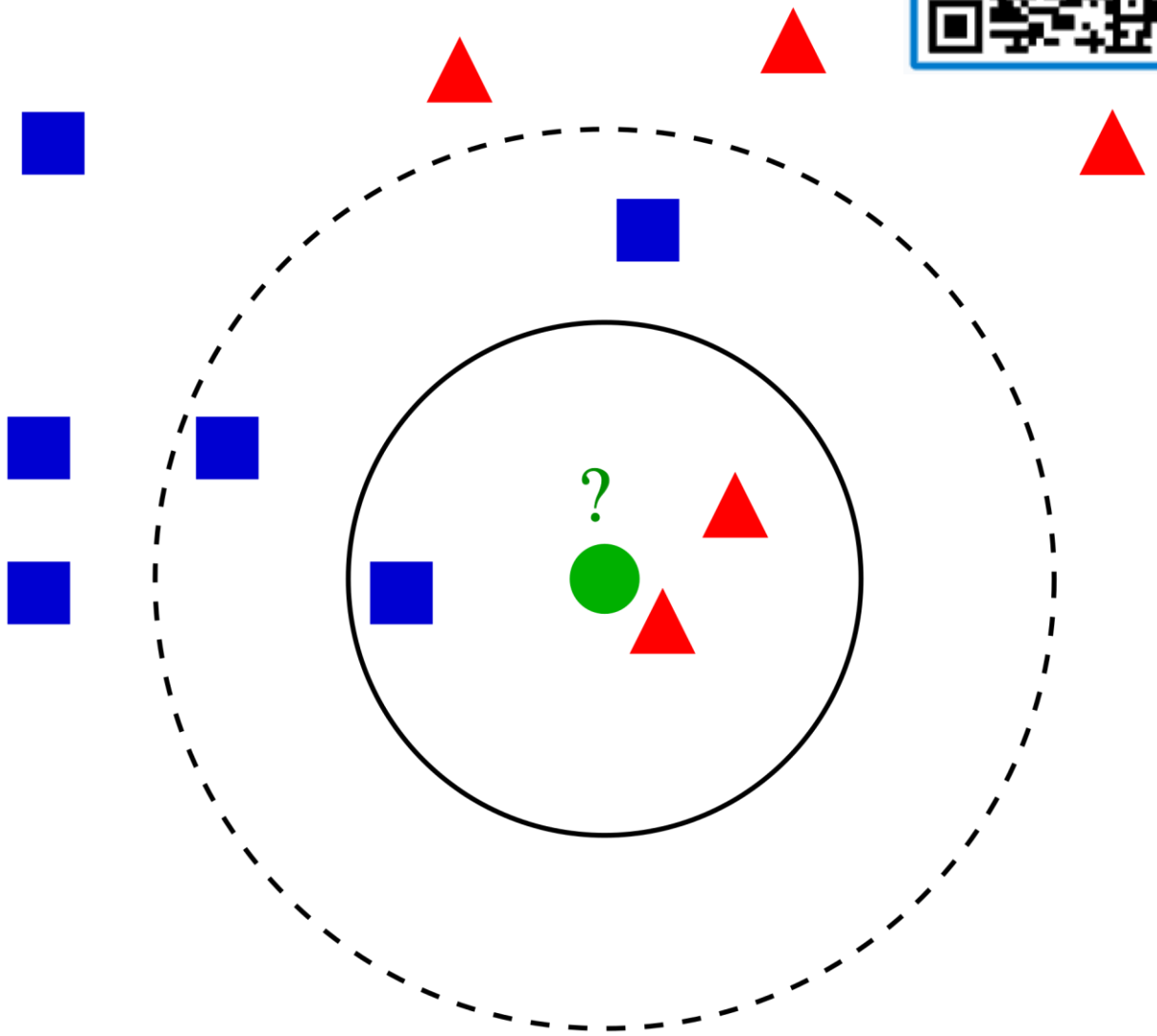
- What happens in very high-dimensional spaces?
  - Points are farther away from each other!

- Let's say we have an n-dimensional unit hypercube
  - We have 1,000,000 points and we are looking for 10 nearest neighbors
  - How big of a neighborhood do we need to look in?
    - 2 dimensions -> 0.003% of the edge length of the unit cube
    - 3 dimensions -> 2% of the edge length of the unit cube
    - 17 dimensions -> 50% of the edge length of the unit cube
    - 200 dimensions -> 94% of the edge length of the unit cube

When poll is active respond at **PollEv.com/492**

Exercise

- Will the new point (green dot) be classified as a blue square or a red triangle with 3 nearest neighbors?
- What about 5 nearest neighbors?
- How do we choose how many neighbors to consider?



Limitations

- What is another limitation of k-NN?
  - All calculations are done at run-time
  - How much training data must be stored?

- There are fixes to speed up k-NN
  - Example: k-d trees (section 19.7.2)
  - Example: Locality sensitive hashing (section 19.7.3)

Moving away from k-NN

- What is a limitation of k-NN?
  - All calculations are done at run-time
  - What does it mean for the utility of this classifier?
  - How much training data must be stored?

- There are fixes to speed up k-NN
  - Example: k-d trees (section 18.8.2)
  - Example: Locality sensitive hashing (section 18.8.3)

- What if we calculate our decision boundaries in advance?
  - Support Vector Machines!

Support Vector Machines

Support Vector Machines

The classifiers exploit three great ideas

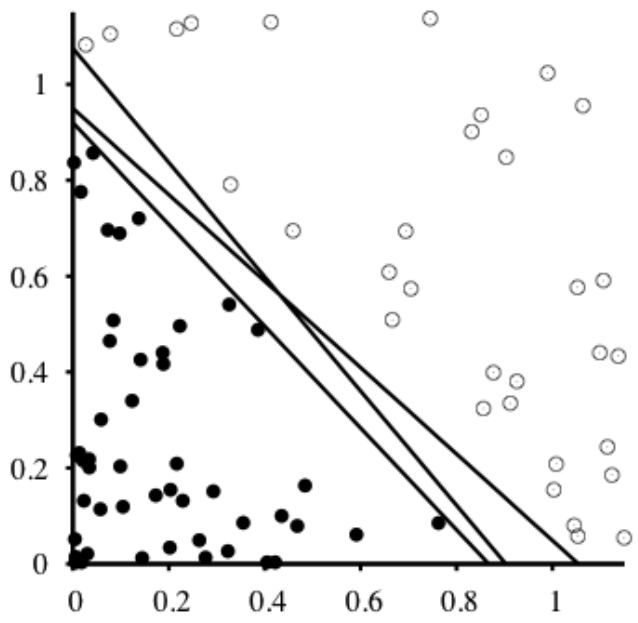
1. Maximum margin separators
   - Robust and focus on critical examples
2. Linear separating hyperplane in a high-dimensional space
   - Can separate data that aren't linearly separable
3. Nonparametric method
   - Only retains a small portion of the training data!

Maximum Margin Separation

- Linear separators

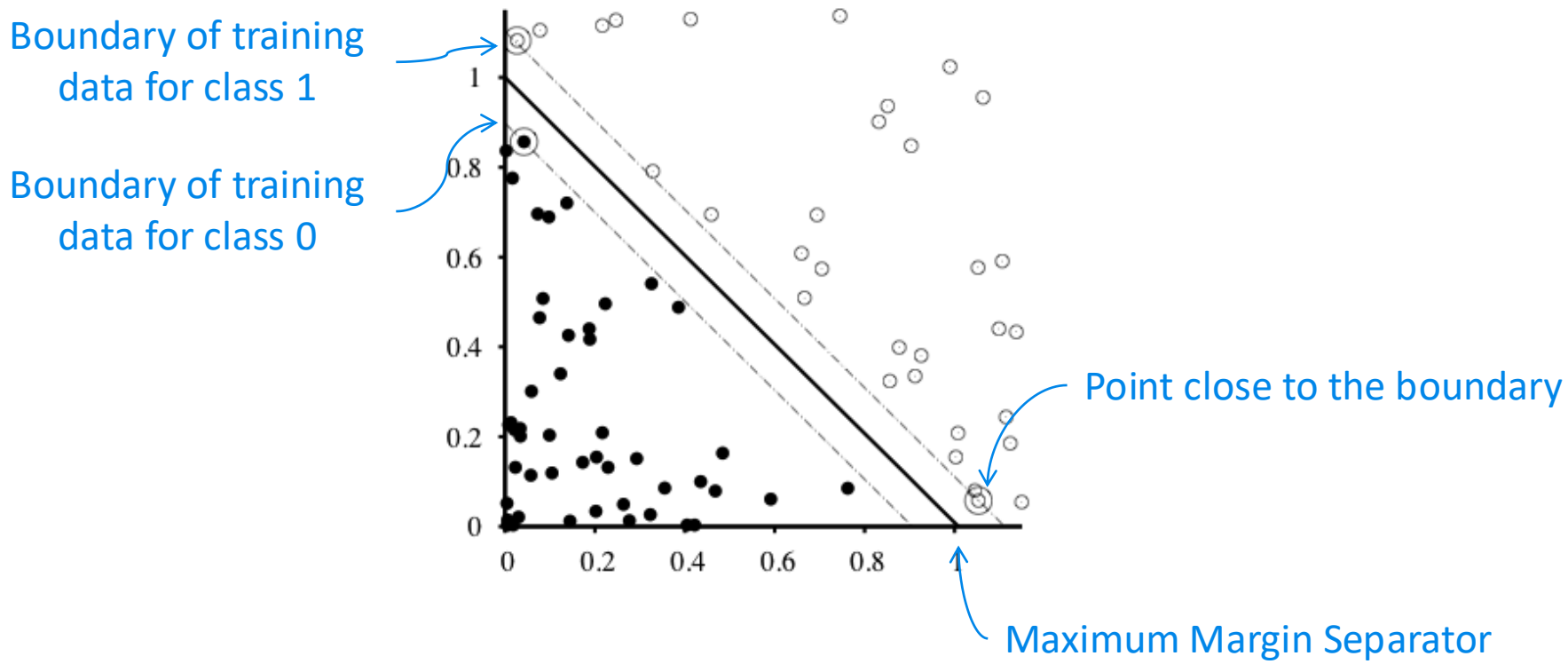
$$h = \sum_i w_i x_i + w_0$$

- How can we choose the best separator?



Maximum Margin Separator

- The best separator has the maximum margin between the decision surface and the examples!



Classification

Feature vector for data point n : $\mathbf{x_n}$
Label for data point n: y_n

- Consider a two-class classification problem:
  - Input feature vectors: $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$
  - Target values: $\{y_1, \ldots, y_N\}$ ---- (positive: y=1, negative: y=-1)

- Train a linear model over the feature vector
  - $f(\mathbf{x}_n) = \mathbf{w} \cdot \mathbf{x_n} + b$
  - $f(\mathbf{x}_n) > 0$ implies $y_n = 1$
  - $f(\mathbf{x}_n) < 0$ implies $y_n = -1$
  - We expect: $y_n\, f(\mathbf{x}_n) > 0$

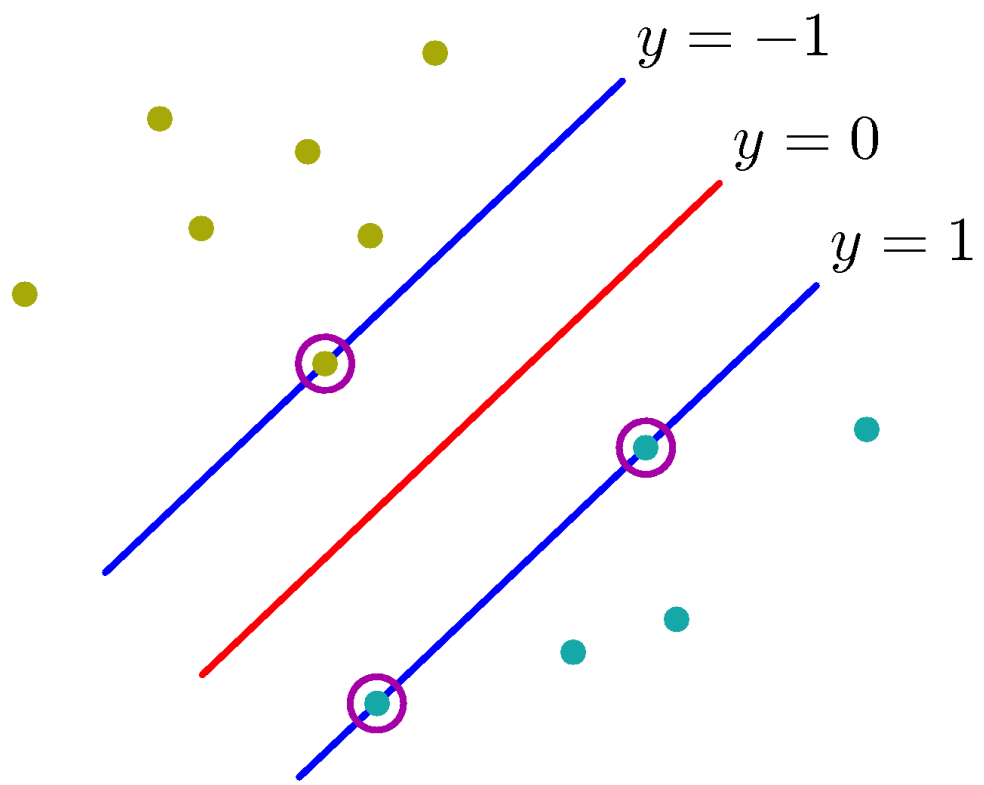
Support Vectors

- The decision surface is:
  $f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = 0$

- The support vectors are the points closest the decision surface

- Find $\mathbf{w}$ so that $y_n(\mathbf{w} \cdot \mathbf{x}_n + b) = 1$ for support vectors

- Only the support vectors determine the decision surface



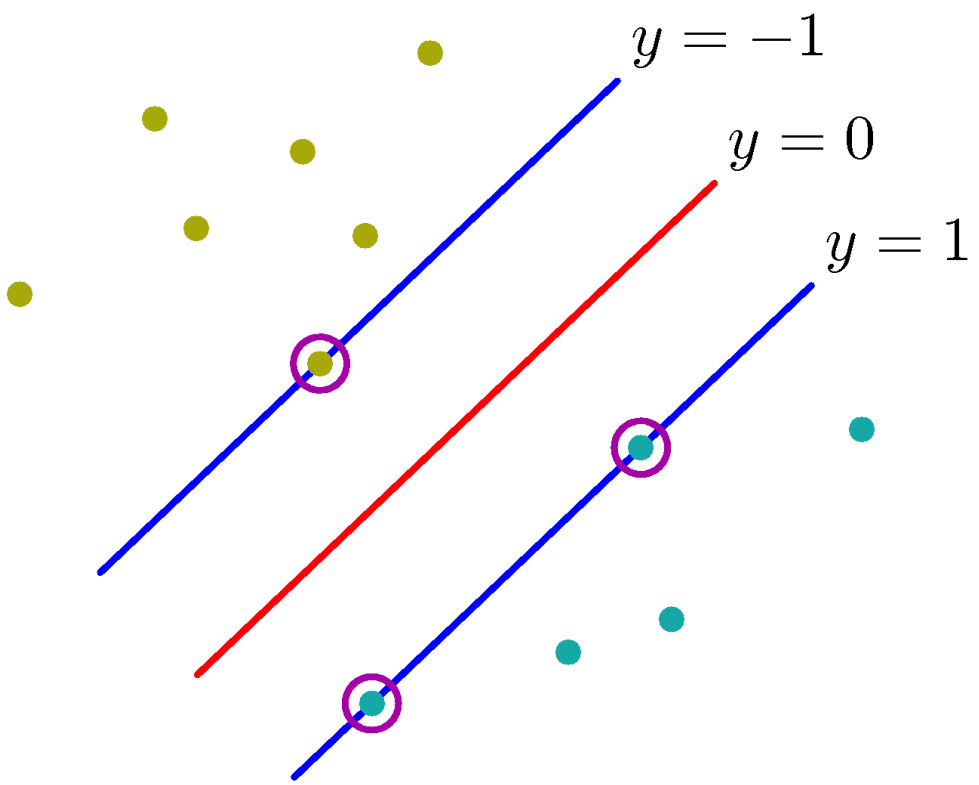
The margin

- Margin: minimum distance from an example to the decision surface
  - For $y_i = 1$: $(\mathbf{w} \cdot \mathbf{x}) + b = 1$
  - For $y_i = -1$: $(\mathbf{w} \cdot \mathbf{x}) + b = -1$

- Distance between the two lines:

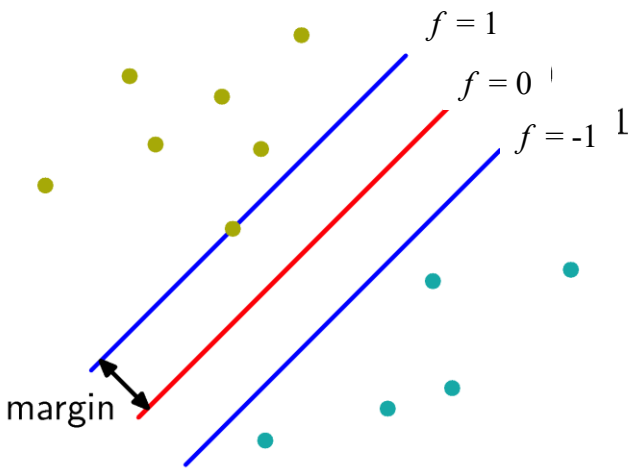
$$\frac{2}{\|w\|}$$

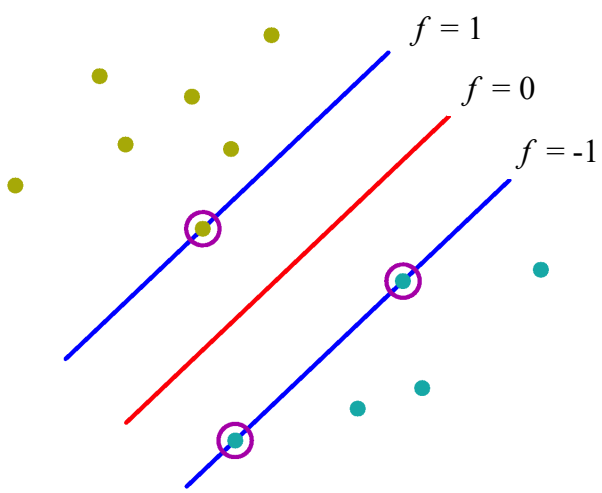
- How can we make the lines far apart?



Maximize the Margin

- Distance to the decision surface is:
  - To maximize margin, maximize $\|\mathbf{w}\|^{-1}$
  - This is the same as minimizing $\|\mathbf{w}\|$ or $\|\mathbf{w}\|^2$
  - Subject to: $y_n\, f(\mathbf{x}_n) \geq 1 \equiv y_n(\mathbf{w} \cdot \mathbf{x}_n + b) \geq 1$





Maximize the Margin via Dual Problem

- Solve this equation for an optimal solution:

$$\underset{\alpha}{\operatorname{argmax}} \sum_{j=1}^{N} \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$

- Constraints:, $\alpha_j \geq 0, \quad \sum_j \alpha_j y_j = 0$

- Then, $\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$

Important Properties:

1. The argmax expression is convex
2. The weights, α, are non-zero only for the support vectors
3. The data enter into the equation as dot products of pairs of points

Then, we can identify the separator

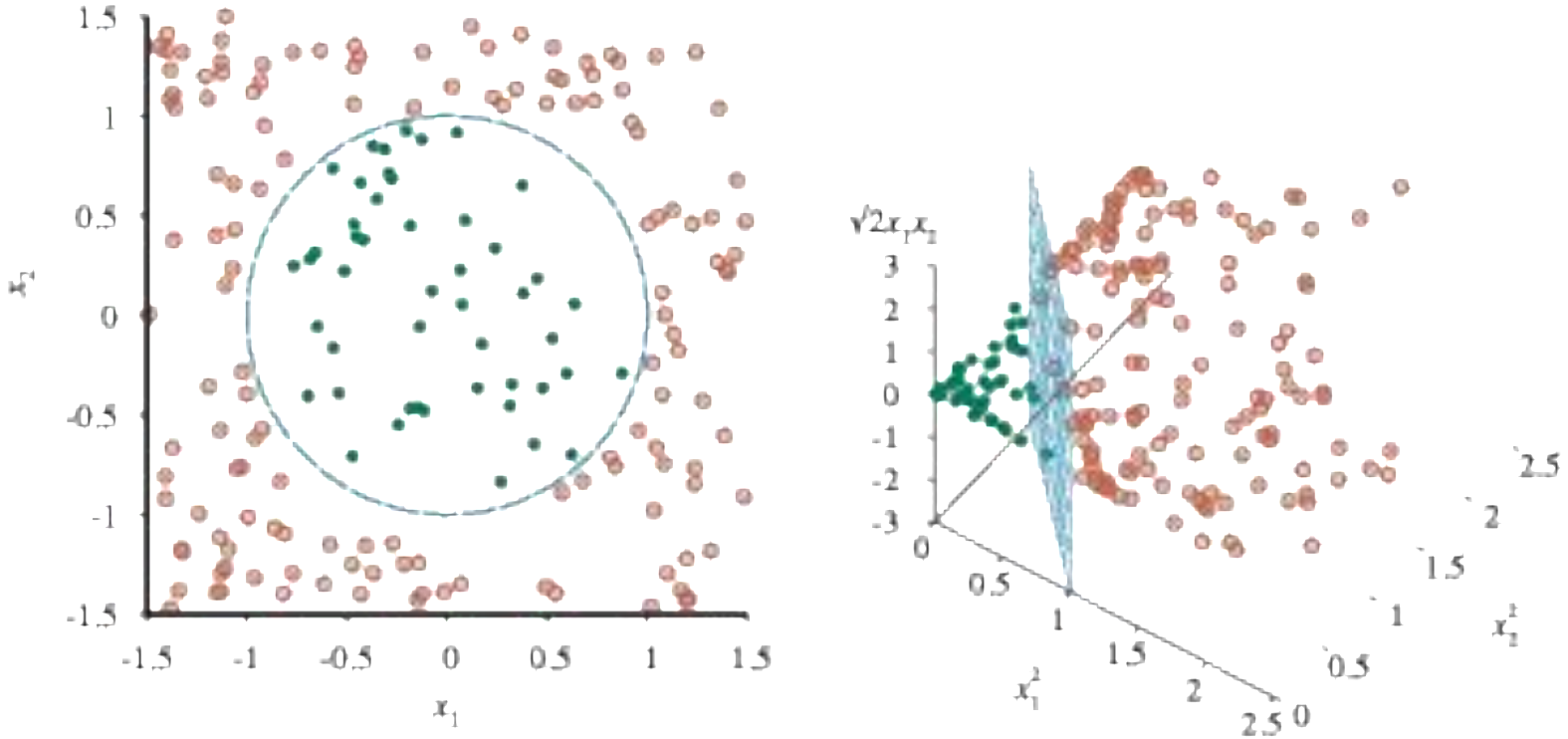
- SVM decision function (separator):

$$h(\mathbf{x}) = \text{sign}(\mathbf{w} \cdot \mathbf{x} - b) = \text{sign}\left(\sum_j \alpha_j y_j (\mathbf{x} \cdot \mathbf{x}_j) - b\right)$$

$$\mathbf{w} = \sum_j \alpha_j y_j \mathbf{x}_j$$

- What if we don't want to capture a linear relationship?
  - We can replace $(\mathbf{x}_j \cdot \mathbf{x}_k)$ with $\left(F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)\right)$
  - Note: this is for each pair of points
  - Example: $\left(F(\mathbf{x}_j) \cdot F(\mathbf{x}_k)\right) = (\mathbf{x}_j \cdot \mathbf{x}_k)^2 = K(\mathbf{x}_j, \mathbf{x}_k)$

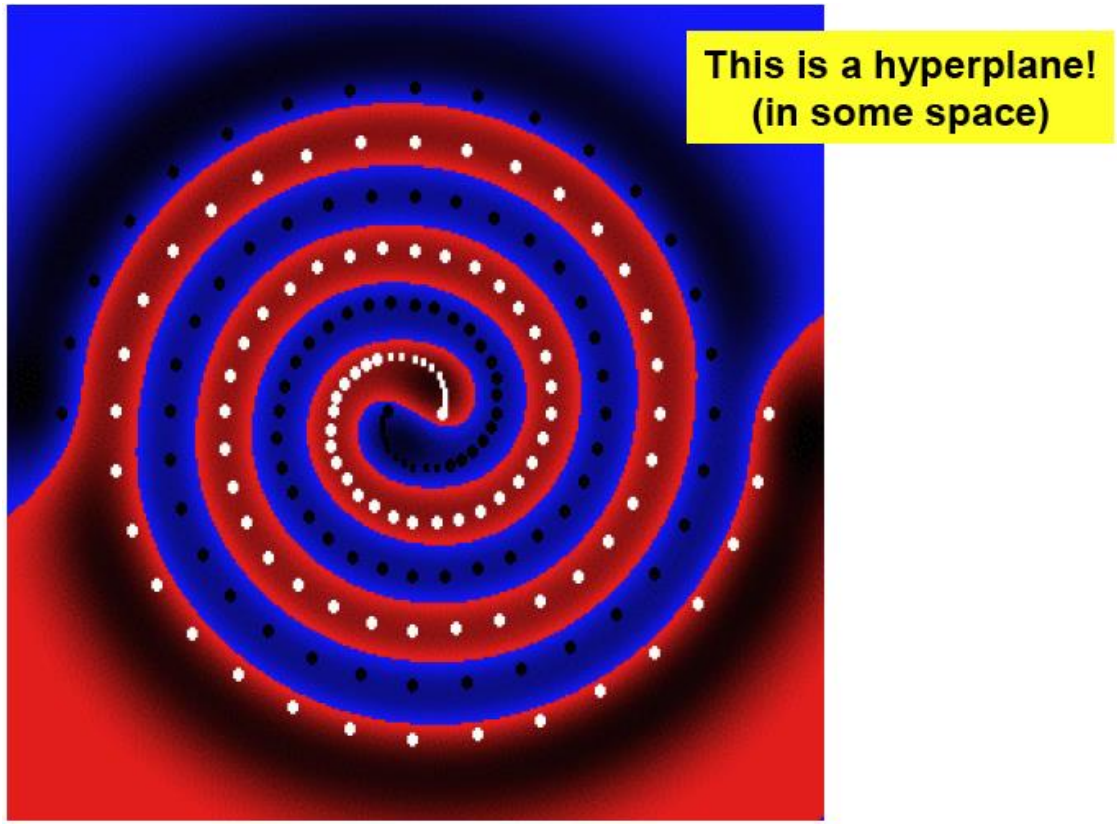
We can transform our feature spaces!



(a) (b)

Why this is Important

- Kernels can separate amazing things!

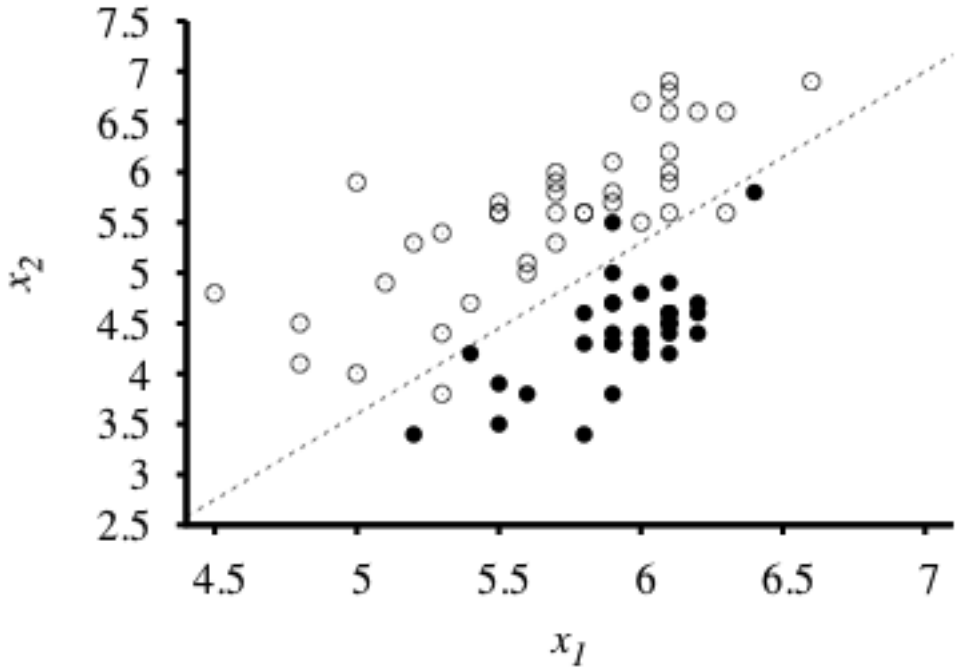


The Kernel Trick

- Many algorithms can be expressed completely in terms of kernels $K(\mathbf{x}_j, \mathbf{x}_k)$ rather than other operations on x
  - You can replace one kernel with another
  - Get a new algorithm that works over a different domain

- Kernel engineering
  - Creates kernels to express different kinds of similarity
  - Sequences, text, images, etc.

What if the sets are not linearly separable?



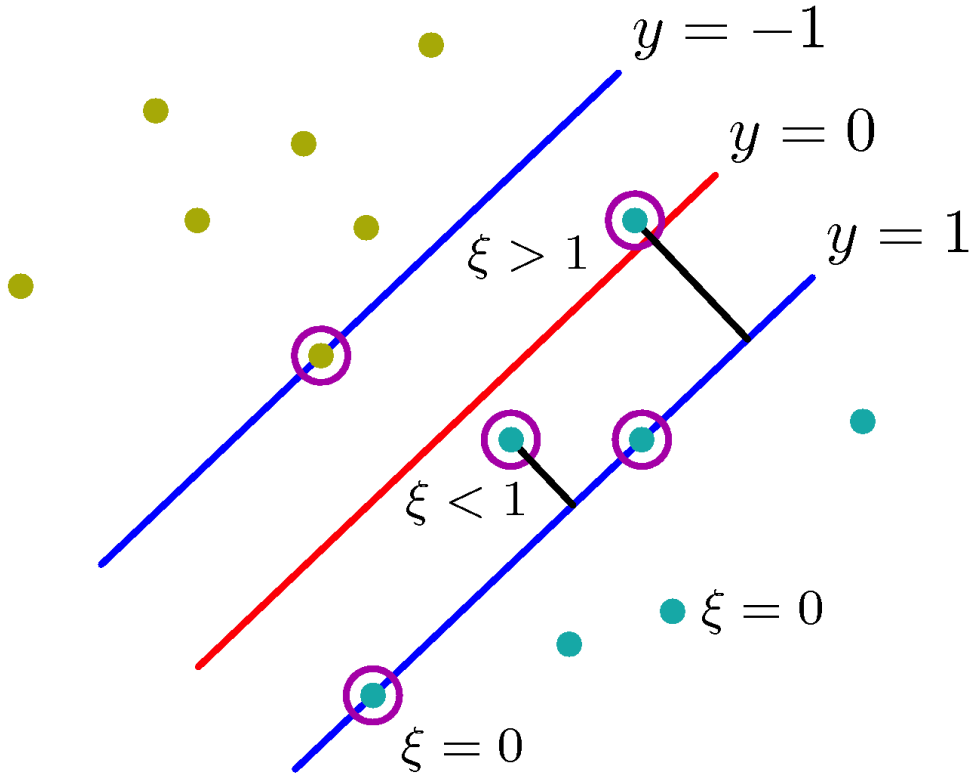
Hard vs. Soft SVM

- Hard SVM requires separable sets:

$$y_n(\mathbf{w} \cdot \mathbf{x}_n + b) \geq 1$$

- Soft SVM introduces slack variables for each data point:

$$y_n(\mathbf{w} \cdot \mathbf{x}_n + b) \geq 1 - \xi_n$$



Soft SVM

- Maximize the margin and also penalize for the slack variables

$$C\sum_{n=1}^{N}\xi_n + \frac{1}{2}||\mathbf{w}||^2$$

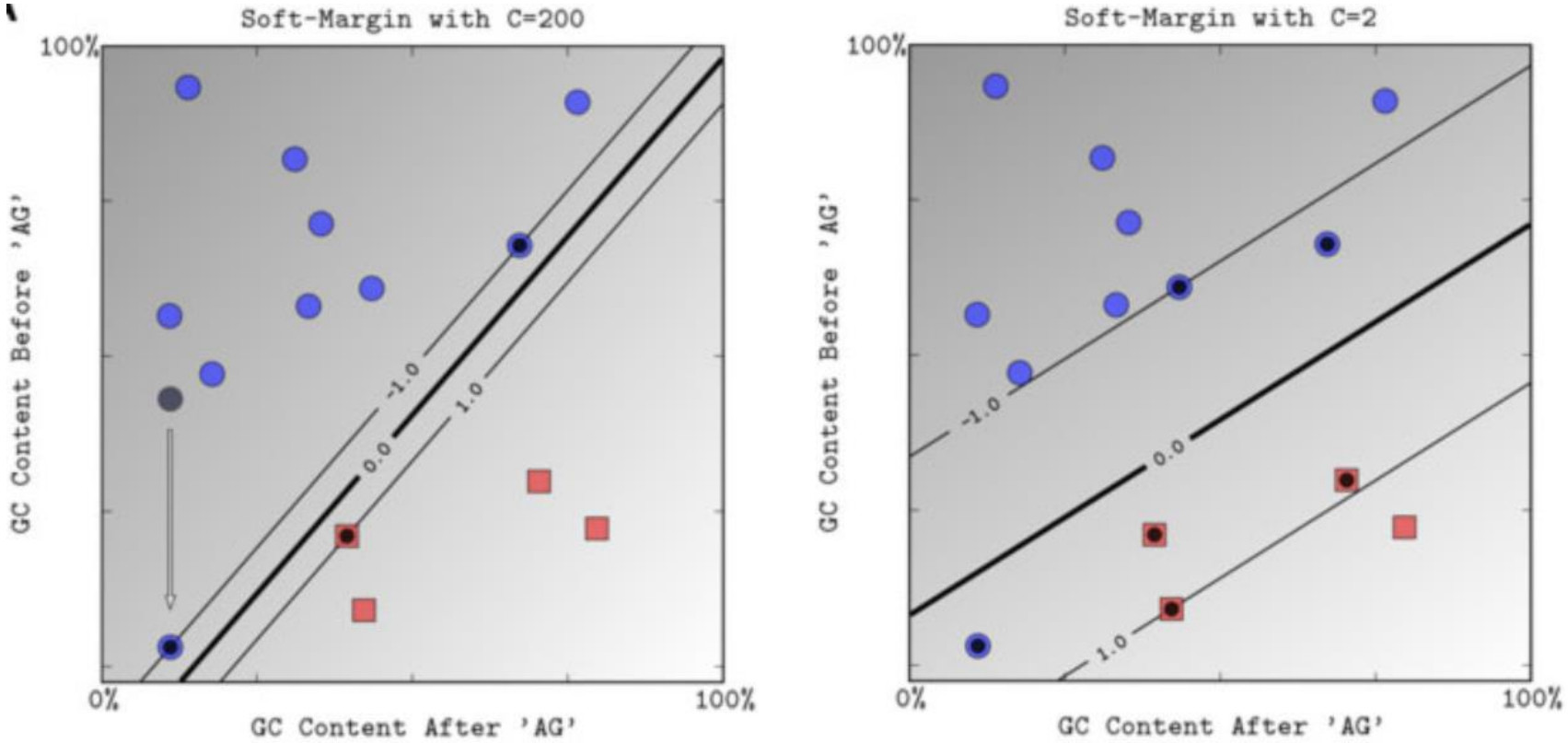
- The support vectors are now those with

$$y_n(\mathrm{w}\cdot\mathrm{x}_n + b) = 1 - \xi_n$$

The SV x_n's are retained – nonparametric!

Soft SVM

- A little slack can give a much better margin



What was that good idea?

- Maximum margin separators are robust
  - Support vectors are the critical examples
- SVMs over feature vectors do nonlinear transformations to make nonseparable data separable in some space
  - Note: Defining a feature space can be difficult
- But a kernel (similarity function) is easier to define and can express the same algorithm!
  - Kernel trick generalizes a kernel algorithm to any kernel